

Architectural Design & Software Design Principles

Student Study Edition · Golden Master design · 10 structured lessons with original course visuals, examples, activities, and study aids.

01 · From Requirements to Architecture

02 · Architectural Design Process

03 · Know Your System Type

04 · System Type → Architectural Style

05 · N-Tier Architecture

06 · Event-Driven & Transformational Architectures

TOPIC 06 · STUDY MAP

Contents

Use this map to move through the topic in the same lesson order as The Educator course pages.

01	From Requirements to Architecture	3
02	Architectural Design Process	7
03	Know Your System Type	11
04	System Type → Architectural Style	14
05	N-Tier Architecture	16
06	Event-Driven & Transformational Architectures	18
07	Persistence, Client-Server & Other Styles	20
08	Custom Architecture & Design Review	22
09	Core Software Design Principles	25
10	Design for the Future	28

TOPIC 06 · LESSON 01 · 1 OF 10 LESSONS

TOPIC 06 · CHAPTER 6 · LESSON 01

From Requirements to Architecture

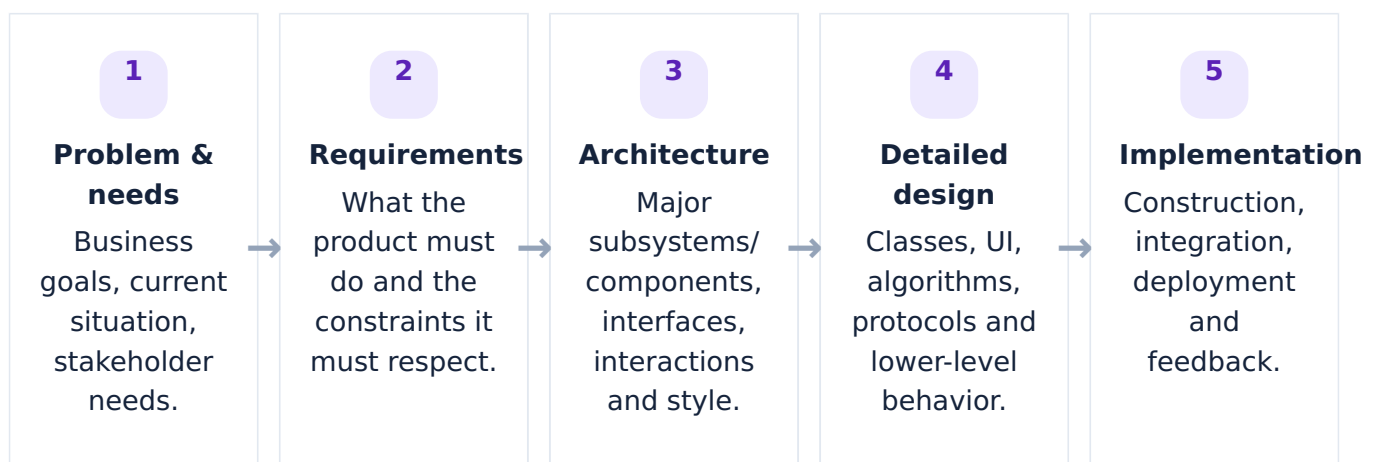
Requirements tell us **what** the product must achieve. Architecture makes the first major decisions about **how the software will be organized** to achieve it.

LEARNING FOCUS

Define software architecture, explain why it matters, and connect requirements, non-functional properties, trade-offs, structure, interfaces, interactions, rationale, and traceability.

PRACTICE LAB · TOPIC 06**Mission 06 — Design the Architecture Before It Designs You**

Architect MediLink across 14 stages: translate requirements into architectural objectives, classify system behavior, choose suitable styles, inspect weak designs, apply cohesion, coupling, KISS and abstraction, and defend the stronger architecture.

THE BRIDGE**What → How**

DEFINITION

Architecture is both a structure and a decision



Software architecture

The architecture of a system or subsystem describes the **style of its high-level structure**, including the interfaces and interactions among its major components.



Architectural design

Software architectural design is the **decision-making process** used to determine the architecture for the system under development.

KEY CONSEQUENCE

DIFFERENT TYPES OF SYSTEMS REQUIRE DIFFERENT DESIGN METHODS AND ARCHITECTURAL STYLES. Architecture is therefore a contextual engineering decision, not a decorative diagram or a universal template.

THREE CLASSIC LENSES

Three ways to remember what architecture captures



Perry & Wolf

Elements + Form + Rationale

Memory hook: **what + how + why.**



Shaw & Garlan

Elements + interactions + composition patterns + constraints.

Architecture explains what the system is built from and the rules shaping how those parts fit together.



Kruchten

Architecture deals with the **high-level structure** of software and involves abstraction, decomposition, composition, style and aesthetics.

COMBINED MENTAL MODEL

Software architecture describes the **HIGH-LEVEL ORGANIZATION of a software system: what major elements exist, how they interact and compose, what patterns and constraints shape them, and why those structural decisions were made.**

WHAT ARCHITECTURE CONTAINS**Structure + interaction + rationale****Elements**

Subsystems, components and other major building blocks.

**Form**

How elements connect, interact and compose into a system.

**Rationale**

Why the design choices were made and which constraints drove them.

**Constraints**

Rules and non-functional properties that shape the structure.

WHY IT MATTERS**Architecture amplifies quality decisions****Performance & efficiency**

Structure, data flow, communication and deployment decisions can strongly affect system performance and efficiency.

**Security**

Boundaries and interfaces determine where protection and trust decisions live.

**Maintainability**

Good decomposition localizes change; poor decomposition spreads it.

**Lifecycle artifact**

Architecture is a primary artifact for conceptualizing, constructing, managing and evolving the system.

✓ Sound architecture

Provides a coherent high-level structure for construction, communication, management and future evolution.

**⚠ Architectural flaws**

Early structural mistakes can propagate across the system and may become serious enough to contribute to **project failure**.

TRADE-OFFS**There is rarely one “best” architecture****Typical trade-offs**

✓ Schedule ↔ Budget

✓ Cost ↔ Robustness

✓ Fault tolerance ↔ Size

✓ Security ↔ Speed

**Engineering discipline**

Record important decisions and their rationale, and maintain traceability back to the requirements that justify them.

MEMORY HOOK

REQUIREMENTS DEFINE THE PROMISE. ARCHITECTURE DEFINES THE HIGH-LEVEL STRUCTURE THAT MAKES THE PROMISE ACHIEVABLE—AND ARCHITECTURAL DESIGN IS THE DECISION PROCESS THAT CHOOSES THAT STRUCTURE.

TOPIC 06 · LESSON 02 · 2 OF 10 LESSONS

TOPIC 06 · LESSON 02

Architectural Design Process

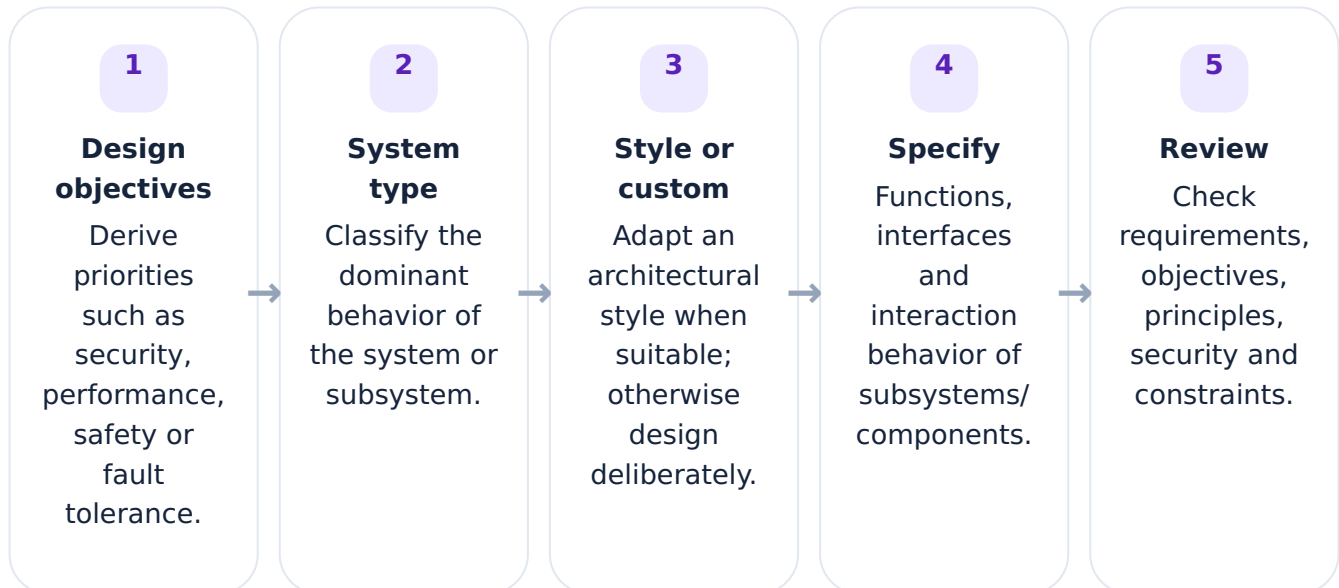
Architecture is a **decision process**: start with objectives, understand the system type, select or adapt a style, specify responsibilities and interfaces, then review and iterate.

LEARNING FOCUS

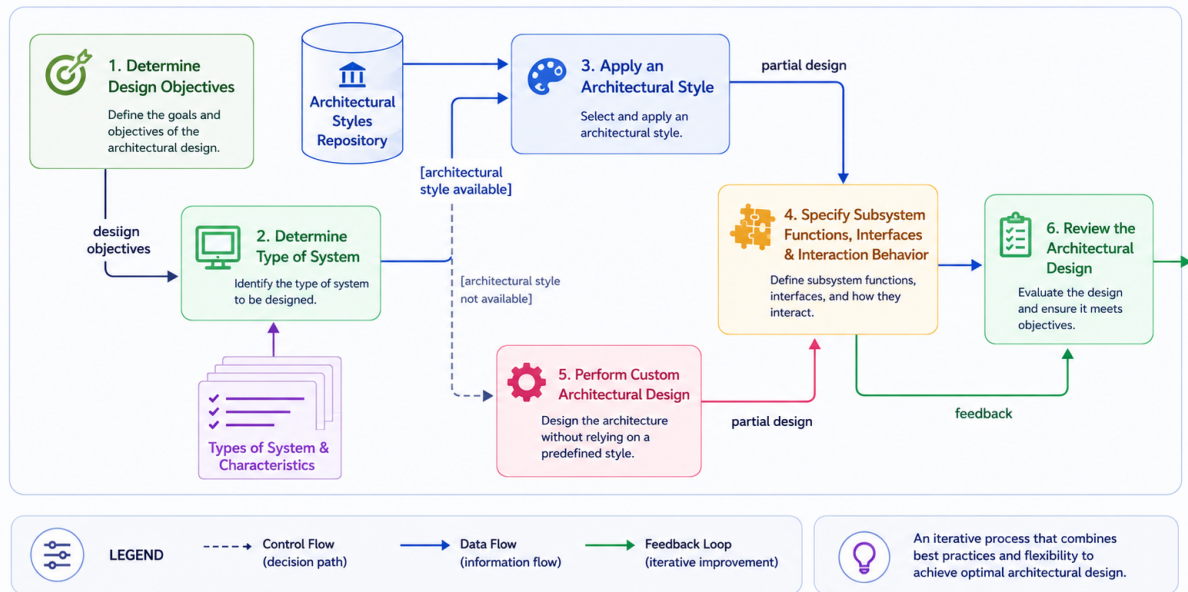
Follow the source architectural-design process and justify decisions using change, COTS, performance, reliability, security, fault tolerance and recovery considerations.

DECISION PIPELINE

Five moves — then repeat if needed



Architectural Design Process



Copyright © The McGraw-Hill Companies, Inc.

Study visual — Architectural Design Process. Use this diagram to see the process as one connected engineering loop rather than isolated checklist items.

1 · Read left-to-right

Start from requirements and architectural objectives before choosing a structure.

2 · Classification matters

System type narrows the architectural styles that are sensible candidates.

3 · Review closes the loop

If the architecture does not satisfy requirements, objectives or design principles, iterate.

ARCHITECTURE DRIVERS

What should influence the decision?



Change & maintenance

How easily must the system evolve without widespread redesign?



COTS

Can commercial off-the-shelf parts reduce effort or impose constraints?

**Performance**

Real-time processing? High transaction volume? Expensive computation?

**Reliability**

How consistently must intended functions work under assumed conditions?

**Security**

How strongly must data and program resources be protected?

**Fault tolerance**

Must operation continue after software problems occur?

**Recovery**

How should the system return to a previous state after a crash?

**Objectives**

Architecture choices should remain traceable to the requirements that created them.

WHEN NO STYLE FITS**Custom architecture is allowed — but not arbitrary****× Weak custom design**

“No style fits, so we draw whatever structure feels convenient.”

**✓ Disciplined custom design**

Use useful partial structures and design patterns, allocate requirements/objectives, specify interfaces and interactions, then review the result.

Mini decision lab

Scenario clue	Architectural concern
Millions of transactions	Performance and capacity become strong design objectives.
Must continue during component failure	Fault tolerance and recovery become architectural drivers.
Sensitive personal data	Security boundaries and protected interfaces matter early.
Rapid business change expected	Ease of change, separation and stable interfaces matter more.

REMEMBER

DO NOT CHOOSE A STYLE FIRST AND INVENT A JUSTIFICATION LATER.

Objectives and system characteristics should drive the choice.

TOPIC 06 · LESSON 03 · 3 OF 10 LESSONS

TOPIC 06 · LESSON 03

Know Your System Type

Before choosing an architecture, classify the dominant behavior. **Different kinds of systems need different design methods.**

LEARNING FOCUS

Distinguish interactive, event-driven, transformational and object-persistence systems using their characteristic flows, actors, state and timing behavior.

FOUR BEHAVIORAL WORLDS

Do not classify by name — classify by behavior

**Interactive System**

Actor-system interaction carries out a business task through a relatively fixed sequence of requests and responses.

fixed-ish sequence

request → response

begins & ends with actor

state tracks progress

**Event-Driven System**

External events arrive without a fixed request order; response depends on the current system state.

random event arrival

state-dependent

many entities

timing constraints

**Transformational System**

A network of processing activities transforms input into substantially different output, usually with little actor interaction during processing.

**Object-Persistence System**

Provides efficient storage, retrieval and update of persistent objects while hiding storage implementation from other subsystems.

batch-like

processing network

often stateless

computation intensive

store/retrieve

huge structured data

little business logic

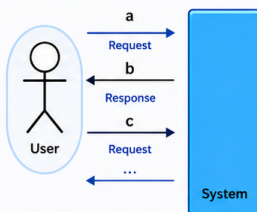
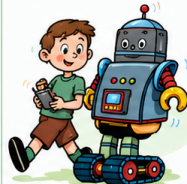
implementation hidden

(a) Interactive subsystem

The user interacts directly with the system.



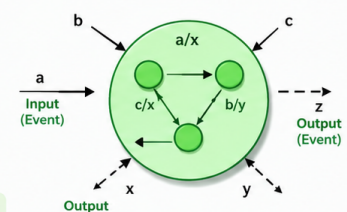
Purpose: Exchange requests and responses between a user and the system.

**(b) Event-driven subsystem**

The system reacts to events that occur in its environment.



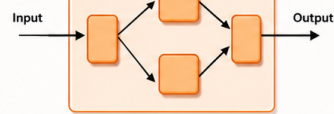
Purpose: Respond to events and produce outputs or trigger actions.

**(c) Transformational subsystem**

The system transforms inputs into outputs.



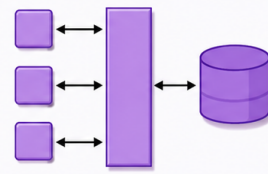
Purpose: Convert input data or materials into desired results.

**(d) Database subsystem**

The system stores, retrieves, and manages data.



Purpose: Organize and provide reliable access to persistent data.



Study visual — System Types. Use the diagram as a classification map. Focus on the dominant stimulus, the role of state, and how much direct actor interaction occurs.

Interactive

Think dialogue and business task: request → response.

Event-driven

Think signals arriving unpredictably; current state changes the response.

Transformational

Think input transformed through processing into a different output.

Persistence

Think durable objects: store, retrieve, update, and hide storage details.

DEEP COMPARISON

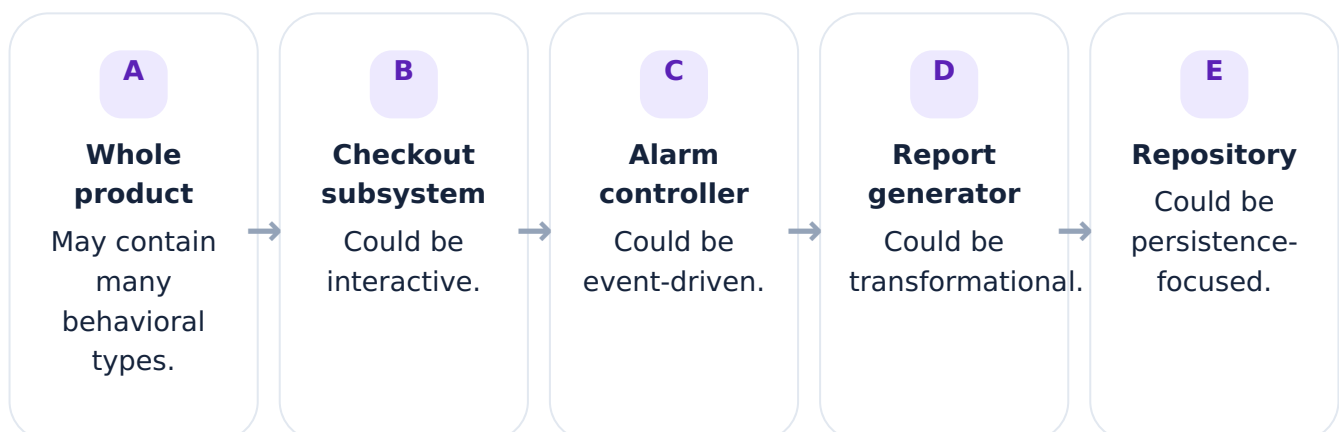
What changes from one type to another?

QUESTION	INTERACTIVE	EVENT-DRIVEN	TRANSFORMAT IONAL	PERSISTENCE
Primary stimulus	Actor requests	External events	Input data	Storage/retrieval requests
Sequence	Relatively fixed	Not fixed	Processing flow	Operation-driven
State role	Tracks business-process progress	Determines response	Usually stateless	Persistent data state
Actor interaction	Frequent	Often devices/software	Little during processing	Supports other subsystems

RECURSIVE VIEW

A large system can contain all four

A system can be a subsystem of a larger system, and one system can contain several subsystem types. Classification is therefore **relative to the subsystem being designed**.



The examples above are study clarifications; the four behavioral definitions and recursive-system rule come from the supplied lecture.

MEMORY HOOK

HUMAN DIALOGUE → INTERACTIVE. REACTIVE SIGNALS → EVENT-DRIVEN. INPUT BECOMES OUTPUT → TRANSFORMATIONAL. DATA SURVIVES → PERSISTENCE.

TOPIC 06 · LESSON 04 · 4 OF 10 LESSONS

TOPIC 06 · LESSON 04

System Type → Architectural Style

Architectural style selection becomes easier when you first understand the system's dominant behavior.

LEARNING FOCUS

Learn the source mappings and understand the selection logic: objectives → system type → suitable style → adaptation or custom design.

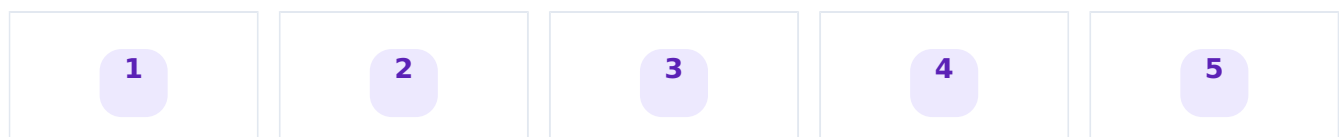
ARCHITECTURE SELECTION MAP

Behavior suggests structure

SYSTEM TYPE	ARCHITECTURAL STYLE IN THE LECTURE	WHY THE FIT MAKES SENSE
 Interactive	N-Tier	Separates presentation, coordination, business behavior, communication and persistence responsibilities.
 Event-Driven	Event-Driven	Events trigger behavior and state influences the response.
 Transformational	Main Program & Subroutines	A controlling flow delegates specialized transformation steps.
 Object-Persistence	Persistence Framework	Storage technology is isolated behind persistence services.
 Client-server	Client-Server	Clients request services from a server through defined communication.
 Distributed, decentralized	Peer-to-Peer	The source maps decentralized systems to P2P.
 Heuristic problem-solving	Blackboard	The source maps heuristic problem-solving to Blackboard.

DECISION LOGIC

A style is a reusable structure — not a rigid template



**AVOID THE MEMORIZATION TRAP****Do not stop at the table****× Weak answer**

“Interactive = N-Tier.”

**✓ Strong engineering answer**

“The subsystem is interaction-heavy, so separating UI, coordination, business behavior and persistence in tiers supports responsibility separation and change control.”

REMEMBER

CLASSIFY FIRST. CHOOSE SECOND. JUSTIFY ALWAYS.

TOPIC 06 · LESSON 05 · 5 OF 10 LESSONS

TOPIC 06 · LESSON 05

N-Tier Architecture

Separate responsibilities into layers so interaction, coordination, domain behavior, communication and persistence do not collapse into one tangled component.

LEARNING FOCUS

Explain why the lecture maps interactive systems to N-Tier architecture and identify the responsibilities of its layers.

VISUAL HOOK · RESPONSIBILITY STACK

Requests move through clear responsibility boundaries

**Graphical User Interface Layer**

Presents information and receives user interaction.

**Controller Layer**

Coordinates requests and connects interaction to application behavior.

**Business Objects Layer**

Represents domain responsibilities and business behavior.

**Network Communication Layer**

Handles communication concerns between distributed parts.

**Database Layer**

Handles persistent storage concerns.

WHY IT FITS INTERACTIVE SYSTEMS

The interaction stays understandable while responsibilities stay separated



Actor-facing

Interactive systems are organized around actor requests and system responses.



Layered responsibility

Each layer has a focused concern instead of every component doing everything.



Interfaces

Requests cross defined boundaries rather than arbitrary direct dependencies.



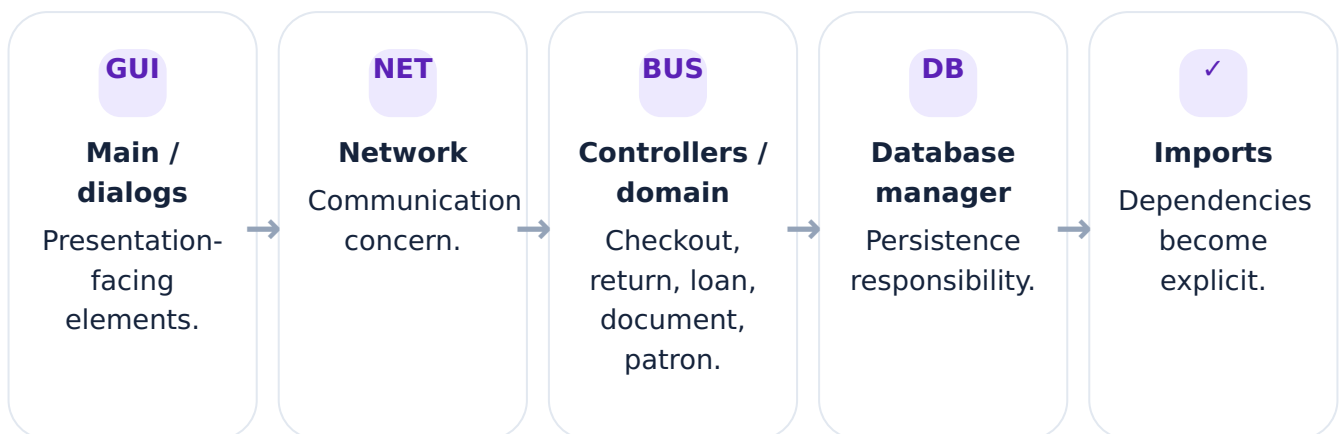
Change control

Stable interfaces can reduce how far a change propagates.

PACKAGE-VIEW CONNECTION

The lecture later shows the architecture as packages

The source N-tier package view separates **GUI, network, business and database** packages and uses «import» dependencies. Public and private elements inside packages show another level of boundary control.



MEMORY HOOK

N-TIER IS NOT “FIVE FOLDERS.” IT IS SEPARATION OF RESPONSIBILITY THROUGH ARCHITECTURAL LAYERS AND INTERFACES.

TOPIC 06 · LESSON 06 · 6 OF 10 LESSONS

TOPIC 06 · LESSON 06

Event-Driven & Transformational Architectures

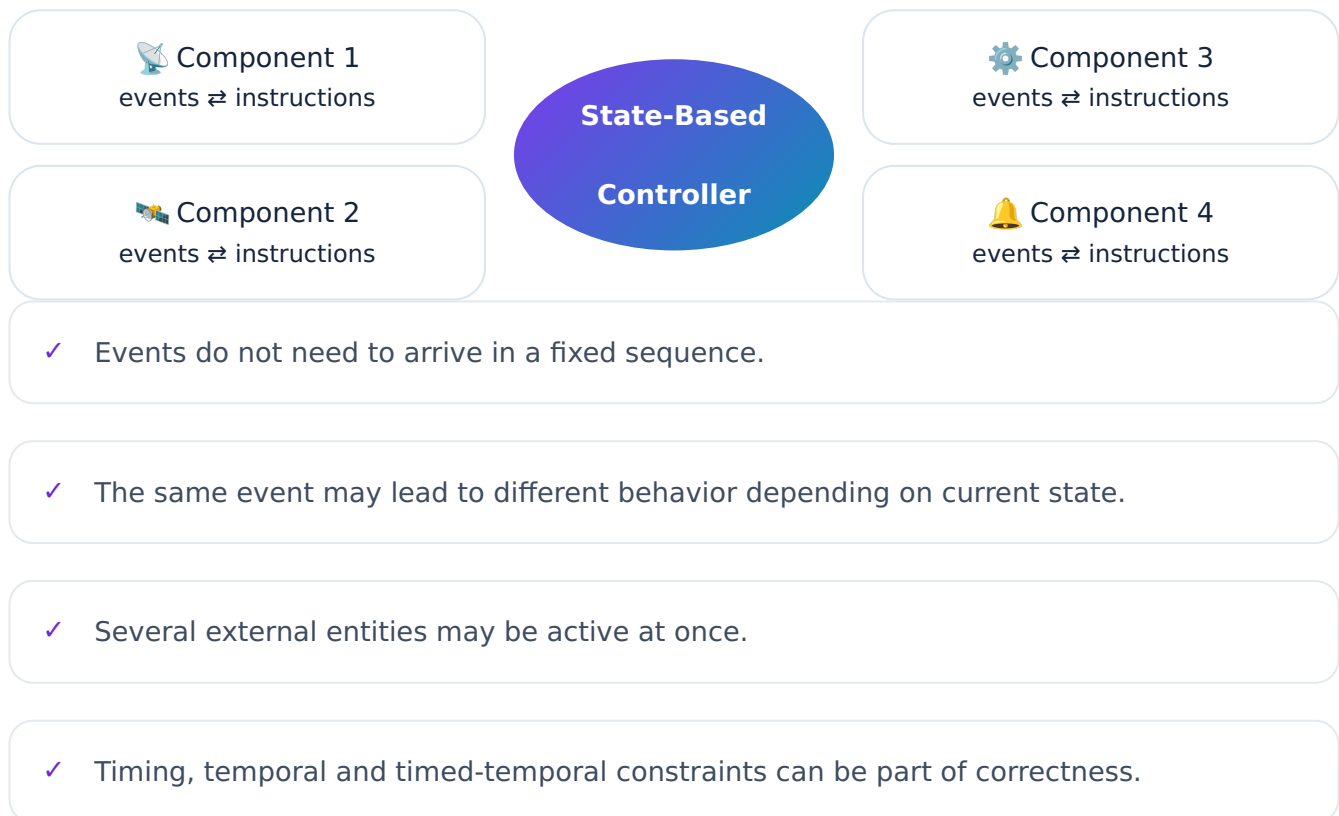
Two very different behaviors need two very different organizing structures: **react to events** versus **transform input into output**.

LEARNING FOCUS

Connect event-driven systems to a state-based event-driven architecture and transformational systems to main-program/subroutine architecture.

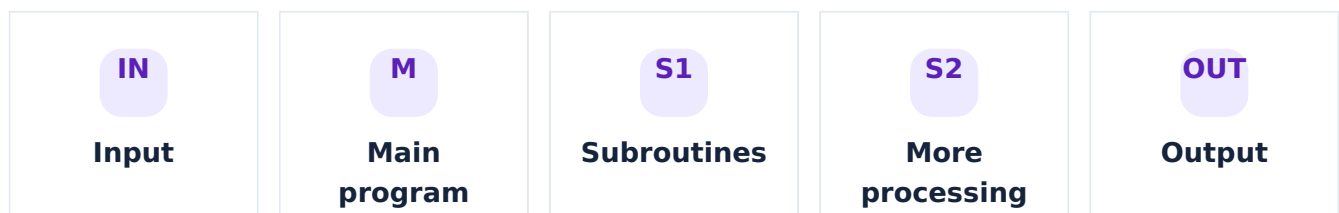
ARCHITECTURE A · EVENT-DRIVEN

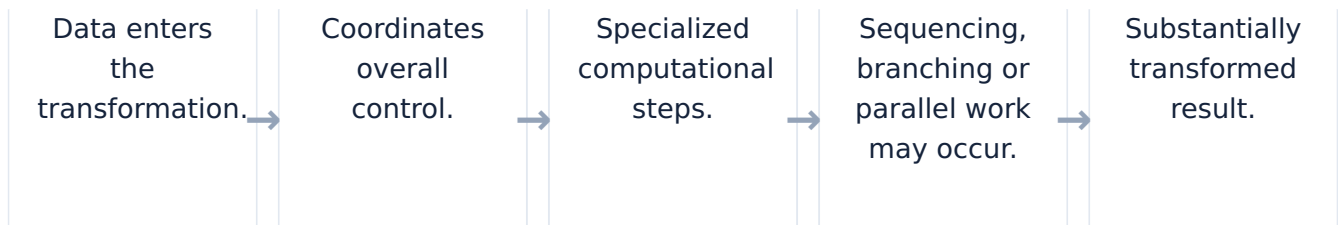
Events arrive; state gives them meaning



ARCHITECTURE B · MAIN PROGRAM & SUBROUTINES

A controlling program delegates transformation steps





The lecture diagram organizes the main program above subroutines connected through «function call» relationships.

DO NOT CONFUSE THEM

Reactive state vs controlled transformation

⚡ Event-Driven

- ✓ Stimulus: external events
- ✓ Order: unpredictable
- ✓ State: central to response
- ✓ External entities: potentially many

⚙️ Transformational

- ✓ Stimulus: input to process
- ✓ Order: processing/control flow
- ✓ State: usually not the central idea
- ✓ Interaction during processing: little or none

STUDY CLARIFICATION

Quick classification test

If the important question is **“What should happen when event X arrives in state Y?”**, think event-driven. If the important question is **“What processing sequence turns this input into the required output?”**, think transformational.

MEMORY HOOK

EVENT-DRIVEN = REACT. TRANSFORMATIONAL = PROCESS.

TOPIC 06 · LESSON 07 · 7 OF 10 LESSONS

TOPIC 06 · LESSON 07

Persistence, Client-Server & Other Styles

Some architectures isolate persistence technology; others organize service relationships across networked components.

LEARNING FOCUS

Explain the object-persistence framework and the detailed client-server example, while correctly recognizing the source mappings for peer-to-peer and Blackboard.

OBJECT-PERSISTENCE FRAMEWORK

Hide storage technology behind a stable manager



Business Objects A · B · C

Use the object-oriented implementation language and should not depend directly on DB-specific details.



DB Manager

Stores/retrieves objects and hides database differences from business objects.



DB Access 1 · 2 · 3

Translate persistence requests toward database-specific communication.



DB 1 · DB 2 · DB 3

Persistent stores addressed through DBMS-specific languages.

This is a concrete architecture example of information hiding: business behavior is protected from database-specific implementation details.

CLIENT-SERVER ARCHITECTURE

Clients request services from a server

«client»

Passenger Check-In 1

«server»

«client»

Passenger Check-In 3

 «client»
Passenger Check-In 2

**Flight
Information
System**

 Communication
«remote procedure call»

The lecture example also introduces UML component notation, input/output ports and stereotypes such as «client» and «server».

OTHER MAPPINGS IN THE SOURCE

Know what is taught — and at what depth

Peer-to-Peer

The supplied lecture maps **distributed, decentralized systems** to Peer-to-Peer architecture, but does not provide the same detailed architectural diagram used for Client-Server.

Blackboard

The source maps **heuristic problem-solving systems** to Blackboard architecture, again without equivalent detailed development in the supplied slides.

CONCEPT CONNECTION

What principle appears repeatedly?

✗ **Direct dependency everywhere**

Business components know DB details; clients know server internals; changes spread widely.



✓ **Stable interfaces**

Managers, servers and ports expose what other components need while hiding implementation decisions.

REMEMBER

PERSISTENCE FRAMEWORK HIDES STORAGE. CLIENT-SERVER ORGANIZES SERVICE REQUESTS. P2P AND BLACKBOARD ARE SOURCE MAPPINGS YOU SHOULD RECOGNIZE WITHOUT INVENTING EXTRA LECTURE DETAIL.

TOPIC 06 · LESSON 08 · 8 OF 10 LESSONS

TOPIC 06 · LESSON 08

Custom Architecture & Design Review

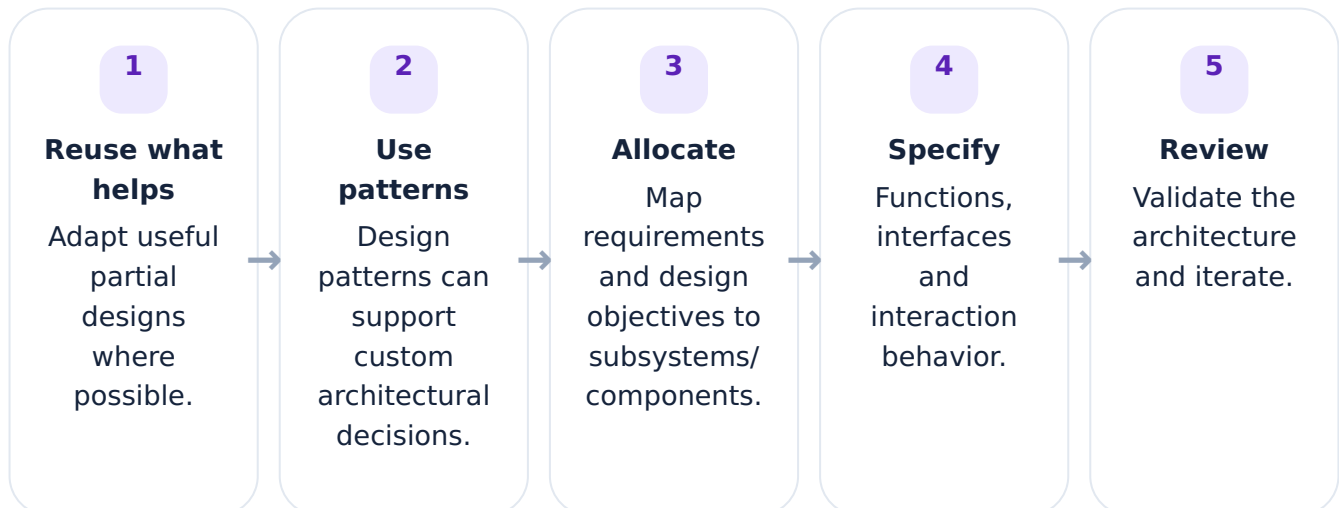
A reusable style is preferred when it fits. When it does not, custom design must still be **traceable, interface-driven, principled and reviewable**.

LEARNING FOCUS

Perform the source custom-design steps, specify subsystem responsibilities and interfaces, and review architecture against requirements, objectives, principles and security constraints.

CUSTOM DESIGN

No style fits perfectly — what next?

**SUBSYSTEM CONTRACT**

Every major component needs three answers



**Interaction**

How does it communicate and coordinate with other parts?

**Traceability**

Which requirements and design objectives justify its existence?

REVIEW GATE

An architecture is not done when the diagram looks clean

**Requirements**

Does the architecture support the required functionality and constraints?

**Design objectives**

Are performance, reliability, changeability or other objectives actually addressed?

**Design principles**

Does the structure respect good separation, cohesion, coupling and information hiding?

**Security**

Are security requirements, constraints and secure-design principles satisfied?

**Iteration**

If the review fails, revise the architecture rather than rationalize the defect.

**Agile guidance**

The source emphasizes working software over comprehensive documentation and a 20/80 “good enough” mindset.

PACKAGE DIAGRAM CONNECTION

Architecture becomes implementation guidance

The lecture’s N-tier package example makes dependencies explicit with GUI, network, business and database packages, «import» relations, and public/private elements. That is the point where architecture begins to guide actual code organization.

MEMORY HOOK

**CUSTOM DOES NOT MEAN IMPROVISED. ALLOCATE → SPECIFY → REVIEW →
ITERATE.**

TOPIC 06 · LESSON 09 · 9 OF 10 LESSONS

TOPIC 06 · LESSON 09

Core Software Design Principles

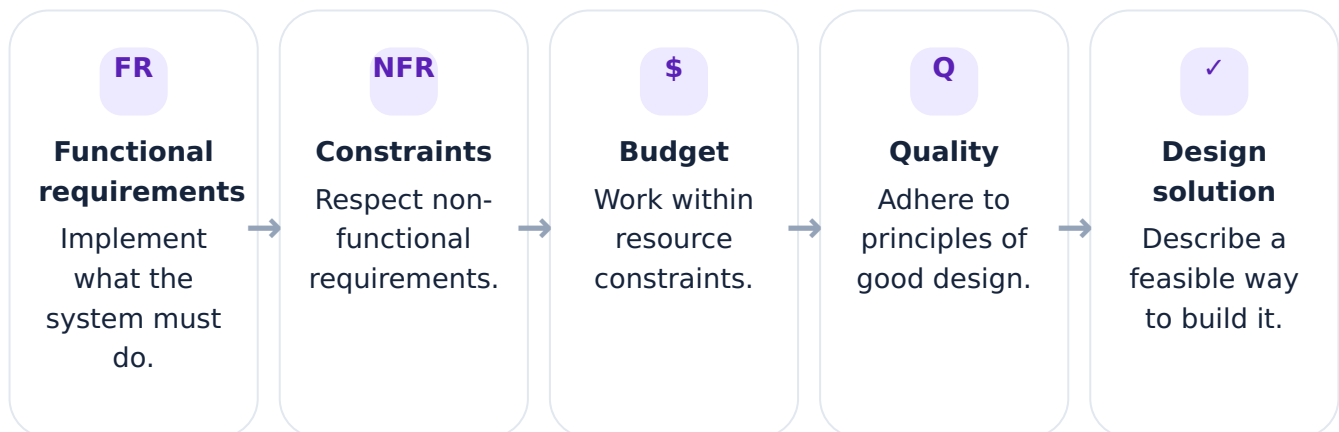
Good design controls complexity and change: **divide the problem, separate concerns, hide details, keep responsibilities cohesive, and reduce dependencies.**

LEARNING FOCUS

Explain the design process and apply divide-and-conquer, design for change, separation of concerns, information hiding, high cohesion, low coupling, KISS and abstraction.

DESIGN ITSELF

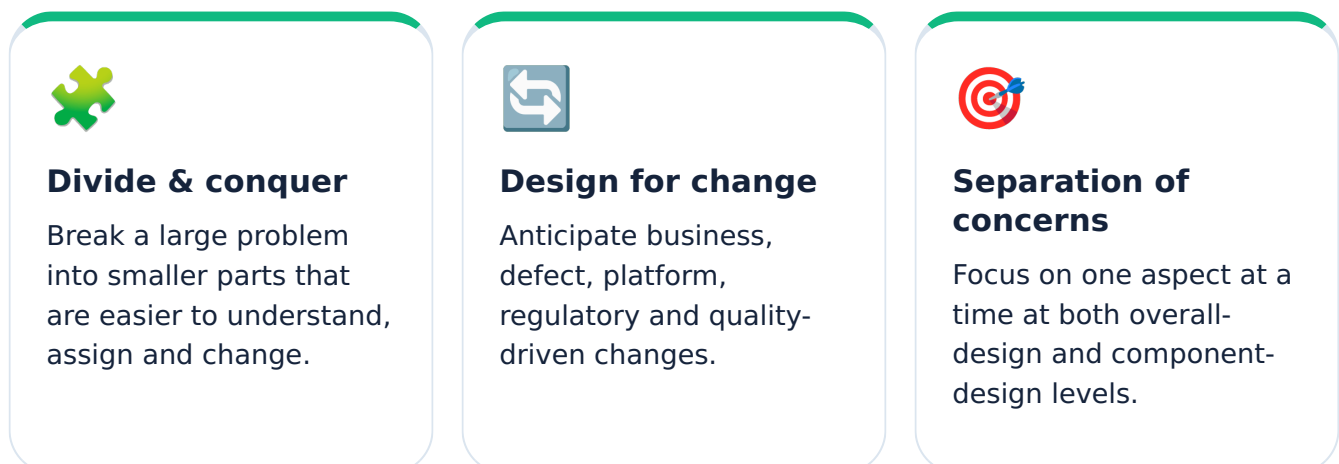
Problem solving under constraints



The lecture distinguishes architecture design, class design, user-interface design, algorithm design and protocol design as different aspects of design.

THE CORE PRINCIPLES

Eight mental hooks





Information hiding

Shield implementation details behind stable interfaces to reduce change impact.



High cohesion

Keep related responsibilities together and unrelated responsibilities out.



Low coupling

Reduce interdependencies so runtime effects and changes do not propagate unpredictably.



KISS

Favor simple, straightforward, easy-to-understand designs.



High abstraction

Hide or defer unnecessary detail so the essence remains understandable.



Encapsulation in the CLO

The supplied CLO names encapsulation, while the detailed slides in this file mainly develop information hiding and private data/stable interfaces rather than a separate encapsulation section.

COHESION VS COUPLING

Inside the module vs between modules



High cohesion

Question: Do the functions inside this module belong together?

Types listed in the source: functional, layer, communicational, sequential, procedural, temporal and utility.



Low coupling

Question: How strongly does this module depend on other modules?

Types listed: content, common, control, stamp, data, routine call, type use, inclusion/import and external.

BAD DESIGN SMELL

Low cohesion + high coupling

× Everything module

Unrelated use cases or methods are grouped together, while many external dependencies leak into the module.

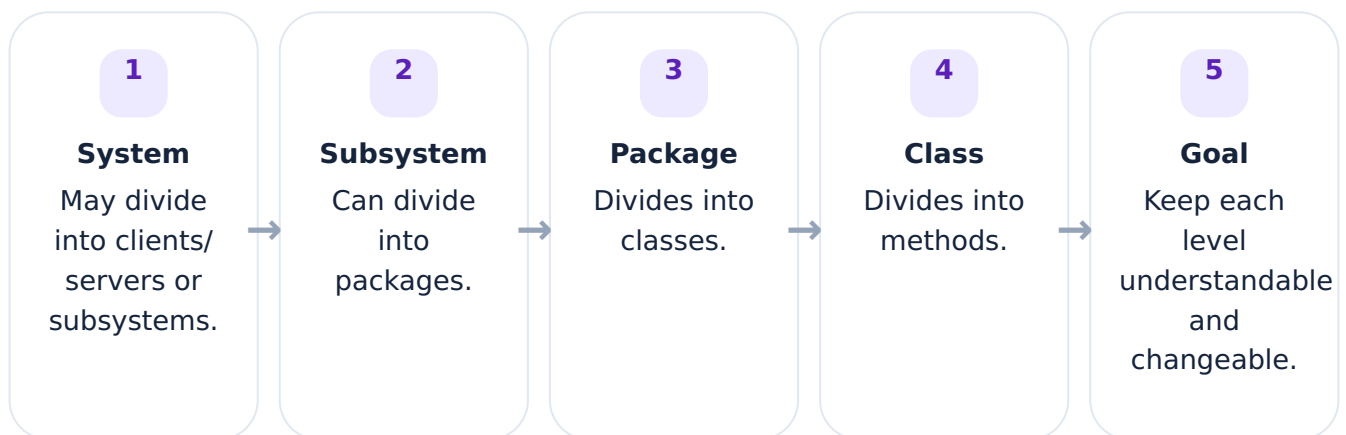


✓ Focused module

Responsibilities are related, interfaces are stable, implementation details are hidden and dependencies are controlled.

HOW SYSTEMS DIVIDE

Decomposition can continue recursively



MEMORY HOOK

COHESION PULLS RELATED THINGS TOGETHER. COUPLING MEASURES WHAT TIES MODULES TO EACH OTHER. INFORMATION HIDING PROTECTS THE BOUNDARY.

TOPIC 06 · LESSON 10 · 10 OF 10 LESSONS

TOPIC 06 · LESSON 10

Design for the Future

A design is not successful only because it works today. Good design prepares for **reuse, change, technology evolution, portability and testing**.

LEARNING FOCUS

Apply the remaining source design principles and synthesize Topic 06 into one architecture-and-design decision model.

FUTURE-FACING PRINCIPLES

Five ways to avoid designing yourself into a corner

**Increase reusability**

Generalize where practical, use good cohesion/coupling/abstraction, add extension hooks and keep design simple.

**Reuse existing designs/code**

Benefit from prior investment in reusable components. The source explicitly says cloning is not good reuse.

**Design for flexibility**

Anticipate change, reduce coupling, increase cohesion, create abstractions and avoid unnecessary hard-coding.

**Anticipate obsolescence**

Avoid fragile dependencies on early, undocumented, environment-specific or poorly supported technologies.

**Design for portability**

Avoid facilities tied unnecessarily to a single environment when broader platform support is required.

**Design for testability**

Make functionality drivable by external test programs without requiring the GUI; automate testing where possible.

REUSE WARNING

Copying is not the same as reuse

× Clone code

The same logic exists in several places; every defect fix or change must be repeated across clones.



✓ Reusable design/ component

One maintained abstraction or component is deliberately designed so multiple contexts can use it.

SOURCE-SPECIFIC GUIDANCE

Concrete rules worth remembering



Avoid fragile technology choices

Early releases, undocumented features and little-used library features increase obsolescence risk.



Think about vendor longevity

The source cautions against specialized technology from suppliers unlikely to provide long-term support.



Prefer standards

Standard languages and technologies with multiple-vendor support can reduce future lock-in.



Testability example

The lecture suggests a Java `main()` method as a simple way to exercise other methods without relying on a GUI.

Topic 06 synthesis · one decision model

Question	What it drives
What must the product do and what constraints matter?	Requirements and architectural objectives
What kind of behavior dominates this subsystem?	System-type classification

Question	What it drives
Is there a suitable reusable structure?	Architectural style or custom design
Who owns which responsibility and interface?	Subsystem/component design
Will the structure remain understandable and changeable?	Cohesion, coupling, separation, hiding, KISS, abstraction
Will it survive future change and be testable?	Reuse, flexibility, obsolescence, portability, testability
Does it still satisfy requirements and objectives?	Architectural review and iteration

CONCEPT CHECK

Can you justify the design?

1

Why should an interactive subsystem often be considered for N-Tier?

2

How do event-driven and transformational systems differ in their dominant control logic?

3

What is the difference between cohesion and coupling?

4

How does information hiding reduce change impact?

5

Why is cloning not considered good reuse in the source?

6

What should be checked during an architectural review?

FINAL MEMORY HOOK

GOOD ARCHITECTURE CHOOSES THE RIGHT STRUCTURE. GOOD DESIGN PRINCIPLES KEEP THAT STRUCTURE HEALTHY AS THE SOFTWARE CHANGES.